

世新大學管理學院資訊管理學系
碩士論文

大小語言模型協作式架構之設計與效能評估-

以多代理人程式測驗評分系統為例

**Design and Performance Evaluation of
Collaborative Architectures for Large and Small
Language Models: A Case Study of a Multi-Agent
Programming Assessment System**

指導教授：胡碩誠 助理教授

研究生：黃子瑜 撰

中 華 民 國 115 年 7 月

世新大學管理學院資訊管理學系碩士（專）班

論文題目：大小語言模型協作式架構之設計與效能評估－
以多代理人程式測驗評分系統為例

研究生：黃子瑜

☐ 本文承蒙下列口試委員審議通過，電子全文於上傳後公開。

☒ 本文承蒙下列口試委員審議通過，電子全文延後 年公開。

口試委員

李永春
胡碩斌
劉育達
李春良

指導教授

胡碩斌

中華民國 115 年 7 月 24 日

誌謝

在撰寫論文的這段期間，我曾感到迷惘，也時常懷疑自己是否能夠順利完成。所幸一路上受到許多人的幫助、支持與鼓勵，讓我得以一步步走到最後。

本論文能夠順利完成，首先衷心感謝胡碩誠老師，在我遭遇困難或對研究方向感到迷惘時，老師總是耐心地引導我釐清問題，並給予寶貴的建議，使我能夠逐步確立研究方向，順利完成本篇論文。也感謝在實驗過程中提供協助與想法的朋友們，你們提出的意見與不同觀點，讓我得以從更多角度思考研究內容。由衷感謝我的家人，在我感到無助與挫折時，謝謝你們始終陪伴在我身旁，無條件的給予我支持與鼓勵，成為我繼續前進的重要力量。同時，也要感謝所有默默關心與支持我的親朋好友們，謝謝你們在這段期間給予我的關懷與鼓勵。最後，誠摯感謝所有曾在這段期間給予我幫助的人，謝謝大家一路以來的陪伴與鼓勵，讓我能夠順利完成論文，走過人生中這段充滿挑戰卻也意義深刻的旅程。

摘要

大型語言模型已具備分析學生程式邏輯、產生評分與文字回饋之能力，可應用於程式測驗自動評分。然而，若固定以雲端大型語言模型執行多次完整推論，模型呼叫量與使用費用將隨評分規模增加而持續累積；改以小型語言模型取代，則可能因模型能力有限，在評分準確度與穩定度上出現落差。本研究提出一套「小型語言模型異質分層與大型語言模型仲裁協作架構」，將程式測驗評分任務區分為行為層、規範層與語言層三個判斷面向，分別交由特性相符之小型語言模型（Qwen2.5-Coder、Gemma 4 E4B、Phi-4）執行初步評分；各層皆進行兩次獨立批改，並以雙次結果之符合準則數是否一致作為分歧判定依據，僅於實際發生分歧之評分層，才交由大型語言模型（Claude Sonnet 5）進行仲裁，其餘已達成一致之層級則不再重複評分。

本研究以完全依賴大型語言模型（對照組 B）、依賴單一小型語言模型並自我仲裁（對照組 C）、以及三層皆由同一小型語言模型執行僅仲裁交由大型語言模型（對照組 D），作為比較基準，並以三份分別對應低分、中等分與高分成績區間之學生考卷進行實驗。實驗結果顯示，對照組 C 於同一次呼叫中同時處理三種性質互異之判斷時，容易產生跨層污染與穩定重複之系統性誤判，且此類誤判可能因雙次結果恆定一致而規避分歧偵測機制；本論文架構將三層評分拆解為獨立任務執行，可降低前述跨層污染問題。與同質分層配置（對照組 D）相比，本論文架構雖未在所有成績區間中皆取得最接近大型語言模型之結果，但整體表現相當或更佳，且批改時間僅約為對照組 D 的一半。與作為主要比較基準之對照組 B 相比，本論文架構於三種成績區間中之差距皆在 3 分以內，且大型語言模型之 Token 使用量減少約 82% 至 90%。

上述結果顯示，透過任務拆解與模型異質化配置，可在維持與大型語言模型相近之評分品質的前提下，有效降低大型語言模型之使用成本；其中，V4 的整體評分結果較接近參照評分，而非在所有情境下皆取得最低誤差。

關鍵詞：大型語言模型、小型語言模型、程式自動評分、多代理人協作、分歧仲裁。

Abstract

Large language models (LLMs) have demonstrated the ability to analyze the logic of student programs, assign scores, and generate textual feedback, making them applicable to automated programming assessment. However, when cloud-based LLMs are repeatedly used to perform complete inference for every submission, the number of model calls and associated usage costs continue to increase with the scale of assessment. Replacing LLMs with small language models (SLMs) may reduce such costs, but their limited capabilities can lead to gaps in scoring accuracy and stability. This study proposes a layered collaboration architecture combining heterogeneous small language models with large language model arbitration, which decomposes the programming assessment task into three judgment dimensions: the behavior layer, the specification layer, and the language layer. Each dimension is assigned to an SLM whose characteristics are aligned with the corresponding task—namely Qwen2.5-Coder, Gemma 4 E4B, and Phi-4—for preliminary assessment. Each layer independently performs two rounds of assessment, and whether the two rounds yield the same number of satisfied criteria serves as the basis for disagreement judgment. Only the layer in which an actual disagreement occurs is submitted to a large language model (Claude Sonnet 5) for arbitration, while layers that have already reached agreement are not reassessed.

Three comparison groups were established as benchmarks: a group relying entirely on an LLM (Control Group B), a group using a single SLM for assessment with self-arbitration (Control Group C), and a group in which all three layers are executed by the same SLM while arbitration is handled by an LLM (Control Group D). Experiments were conducted using three student examination papers corresponding to low-, medium-, and high-score ranges. The experimental results show that when Control Group C processes three fundamentally different types of judgment within a single model call, it is prone to cross-layer contamination and consistently repeated systematic misjudgments; because such misjudgments may remain identical across both rounds, they can evade the disagreement detection mechanism. By decomposing the three assessment layers into independent tasks, the proposed architecture reduces this cross-layer contamination problem. Compared with the homogeneous layered configuration

in Control Group D, the proposed architecture did not achieve results closest to the LLM in every score range; nevertheless, its overall performance was comparable or superior, and its assessment time was only about half that of Control Group D. Compared with Control Group B, the primary benchmark, the proposed architecture maintained score differences of fewer than three points across all three score ranges and reduced LLM token usage by approximately 82% to 90%.

These findings indicate that task decomposition and heterogeneous model allocation can effectively reduce the cost of using LLMs while maintaining assessment quality comparable to that of an LLM-based approach. Overall, V4 produced assessment results that were closer to the reference scores, although it did not achieve the lowest error in every scenario.

Keywords: large language model, small language model, automated programming assessment, multi-agent collaboration, disagreement arbitration.

目錄

誌謝.....	i
摘要.....	ii
Abstract.....	iii
目錄.....	v
圖目錄.....	vii
表目錄.....	viii
一、緒論.....	1
1-1 研究背景.....	1
1-2 研究動機.....	2
1-3 研究目的.....	3
1-4 論文架構.....	3
二、文獻探討.....	5
2-1 語言模型於程式自動評分之應用.....	5
2-2 小型語言模型的界定、部署特性與訓練差異.....	7
2-3 結構化評分與多代理人任務分工.....	10
2-4 大小模型協作、仲裁機制與成本考量.....	12
2-4-1 模型路由與串接式協作.....	12
2-4-2 分歧檢查與大模型仲裁.....	14
2-4-3 成本考量.....	15
三、研究方法.....	17
3-1 系統架構圖.....	17
3-2 題目結構化與評分準則設計.....	18
3-2-1 三層評分準則定義.....	18
3-2-2 題型分類與權重設計.....	21
3-2-3 關鍵字設計.....	22
3-3 模型選擇與設計和仲裁機制.....	23
3-3-1 小型語言模型角色配置.....	23
3-3-2 小型語言模型提詞設計.....	25
3-3-3 大型語言模型仲裁選擇設計與仲裁流程.....	26
3-4 結果整合與計分方式.....	27
四、研究結果與分析.....	30

4-1 結構化評分下單一模型配置之表現.....	31
4-2 任務拆解與模型異質化之比較.....	34
4-3 整體降低成本之驗證.....	36
4-4 本章小結.....	38
五、結論與未來展望.....	41
參考文獻.....	43
附錄 1、題型與題目範例.....	46
附錄 2、結構化題目範例.....	50
附錄 3、各模型提詞設計.....	53
SLM1 行為層	53
SLM2 規範層	54
SLM3 語言層	56
LLM 仲裁.....	59

圖目錄

圖 1、系統架構圖.....	18
----------------	----

表目錄

表 1、三個評分層次之判斷定義.....	20
表 2、題型分類權重配置.....	22
表 3、小型語言模型之角色配置.....	25
表 4、對照組 B 與對照組 C 之結果比較	32
表 5、對照組 C 與對照組 D 與實驗組 V4 之結果比較	35
表 6、對照組 B 與實驗組 V4 之比較	37
表 7、題型與題目範例.....	46
表 8、結構化題目範例.....	50

一、緒論

1-1 研究背景

近年來，大型語言模型（Large Language Models, LLMs）逐漸被應用於程式設計教育，相關應用包括程式碼生成、錯誤診斷、學習回饋及程式作業評分。傳統自動程式評分系統通常透過測試案例執行並比對輸出結果，或以靜態分析技術將學生提交之程式與參考解答或正確範例進行比對[18]；大型語言模型則可根據題目、評分準則與學生程式內容產生評分及文字說明，使自動評分不再只依賴程式輸出或是否通過測試案例。

已有研究顯示，大型語言模型的評分結果可與人工評分相近，具備支援程式作業評分與學習回饋的可行性[24]；但不同模型之間的評分結果仍存在落差，顯示模型選擇會影響評分結果[6][16][25]。除了評分結果因模型選擇而異之外，大型語言模型通常仍需要較高的運算資源，並多透過雲端服務提供推論功能。當其應用於大量學生作業、高頻率的即時回饋時，模型呼叫次數、文本傳輸量與使用成本將隨評分規模增加。因此，在需要頻繁執行推論的教育場景中，系統的運算需求與經濟成本是不可忽視的設計考量。

基於上述成本與部署考量，小型語言模型（Small Language Models, SLMs）因參數規模較小、硬體需求低，且可透過量化技術部署於個人電腦或邊緣裝置，逐漸受到研究界關注。然而，單純依賴小型模型或大型模型，皆可能受限於各自的資源或效能限制，因此結合不同規模模型的協作架構，已成為近期語言模型系統設計的研究方向之一[9]。大型語言模型具有較強的理解與推理能力，但需要較高的運算資源及使用成本；小型語言模型則具有本地部署及較低資源需求等特性。如何依據實際任務需求，安排大型與小型語言模型所負責的工作，

已逐漸成為語言模型實際應用中的重要議題。

1-2 研究動機

將小型語言模型應用於程式評分，雖能透過本地端部署降低依 Token 計費的雲端 API 費用，並解決大量重複評分的成本問題；然而，程式評分涉及邏輯推理與語意分析，且判斷錯誤會直接影響給分結果。小型語言模型受限於模型規模與理解能力，在面對結構複雜、寫法多元的學生程式時，往往存在較高的評估誤差風險。為了在降低成本的同時確保輸出品質，其他應用領域已出現「動態路由 (Dynamic Routing)」與「模型級聯 (Cascading)」的大小模型協作模式，其作法可作為程式評分設計的參考方向：部分研究讓小型模型依信心程度決定是否轉交大型模型處理[8][28]，亦有研究採用多個本地端小型模型共同評估並取得共識、僅在共識度不足等少數疑難情況時，才將任務升級 (Escalate) 交由雲端大型模型處理[11]，兩者皆能在維持一定表現的前提下大幅降低推論成本。

上述協作機制雖在一般推理或問答任務中取得成效，但程式評分具有獨特的挑戰。以常見的程式設計測驗為例，同一道題目的學生解法可能使用不同的迴圈結構或條件判斷式達成相同輸出結果；評分時還須進一步判斷是否依照題目規定完成，以及程式語法本身是否正確。這三種判斷各自涉及不同的評分邏輯，難以由單一模型或單一策略一次到位處理。如何針對這種多面向的評分需求設計合適的協作架構？當小模型因能力無法應付時，是否有救援機制補足？在引入協作架構後，能否在減少成本的同時，維持與大型語言模型相近的評分結果？這些問題在現有文獻中仍缺乏系統性的探討。

因此，本研究動機在於探討小型語言模型是否能承擔程式測驗中的主要評分工作，並在評分結果出現分歧時由大型語言模型進行進一步判斷，分析協作

架構與大型語言模型評分結果之差異，並評估能否降低大型語言模型的使用量與推論成本。

1-3 研究目的

基於上述動機，本研究從架構設計、評分結果與推論成本三個面向進行比較，具體訂定以下研究目的：

1. 設計一套結合小型語言模型評分與大型語言模型仲裁之協作式程式評分架構，以支援程式測驗的自動化評量。
2. 比較本研究所提出之大小語言模型協作架構與大型語言模型評分架構的評分結果，分析兩者之間的差異程度。
3. 比較所提出之大小語言模型協作架構與大型語言模型評分架構的推論成本，評估其是否能減少大型語言模型的使用量與整體費用。

1-4 論文架構

本研究旨在設計並驗證一套結合小型語言模型分層評分與大型語言模型分歧仲裁之程式測驗自動評分架構，以在維持評分品質的前提下，降低大型語言模型之使用成本。本論文共分為五章，各章內容概述如下。第一章為緒論，說明大型語言模型應用於程式自動評分之研究背景與現況，並依此提出本研究之動機、目的。第二章為文獻探討，回顧語言模型於程式自動評分之應用、小型語言模型之界定與訓練差異、結構化評分與多代理人任務分工，以及大小語言模型協作、分歧檢查與仲裁等相關研究，並據以確立本研究之定位與文獻缺口。第三章為研究方法，說明本論文所提出之「小型語言模型異質分層與大型語言模型仲裁協作架構」，包括結構化評分準則與資料前處理方式、三個小型語言模型之角色配置與選型依據、雙次批改與分歧仲裁機制、結果整合與計分方式，以及系統之完整運作流程。第四章為研究結果與分析，透過對照組 B（單一大

型語言模型)、對照組 C (單一小型語言模型)、對照組 D (同質分層) 與實驗組 V4 (本論文架構) 之比較，驗證單一模型之限制、結構化之效益，以及模型異質化配置於評分品質與推論成本上之表現。第五章為結論與未來展望，總結本研究之主要發現與貢獻，並提出後續研究方向之建議。

二、文獻探討

本章針對語言模型應用於程式自動評分所涉及的研究進行整理。首先說明傳統程式自動評分的處理方式，以及大型語言模型在程式評分中的應用與限制；其次整理小型語言模型的界定、部署條件與模型間差異；接著探討結構化評分準則與多代理人任務分工；最後說明大小語言模型協作、重複推論、分歧檢查與條件式模型介入等方法。

2-1 語言模型於程式自動評分之應用

程式作業的自動評分通常透過編譯與執行學生程式，再以預先設計的測試案例比較實際輸出與預期結果。此類方式能有效檢查程式是否完成指定輸出，但若學生的程式僅部分正確，或未符合題目所要求的特定演算法、程式結構與程式碼品質，僅依據測試結果進行評分，可能不足以準確判斷應給予的部分分數及具體扣分原因。

Nagakalyani 等人針對程式設計入門課程提出 TA Buddy，以 Code Llama 為基礎，並使用已評分的學生程式資料進行微調。該工具接收題目敘述、學生程式碼與評分標準，依不同準則產生建議分數，再由助教接受或修改模型結果。研究者比較 TA Buddy 輔助評分與完全由人工進行的評分流程，結果顯示，使用 TA Buddy 可使評分時間減少 24%，兩種評分方式的分數一致率為 90%。此研究說明，語言模型可讀取學生原始碼並依評分準則提供分項建議，但最終結果仍保留由人工複核的機制[19]。Cisneros-González 等人提出 JorGPT，將可編輯的評分規準提供給多個大型語言模型，使模型分別判斷學生程式的邏輯、語法與其他評分面向，並保留教師介入評分流程的機制[10]。Pathak 等人則比較題目特定評分規準與不依題目調整的通用評分規準，並以資料結構與演算法及物件導向程式設計的學生作答進行測試。研究結果顯示，題目特定評分規準在程式邏

輯判斷與回饋方面，較能對應專家評分[21]。上述研究顯示，運用語言模型進行程式評分時，不宜僅以「功能是否正確」或「程式碼是否良好」等概括性描述作為評分依據，而應根據個別題目的內容與要求，建立明確且可判定的題目特定評分標準。然而，題目特定評分標準並不代表每一道題目皆須採用完全不同的評分類別，而是可先建立一致的整體評分架構，再依各題內容調整各評分面向下的判斷項目。例如，可將功能與演算法、題目指定要求，以及語法與型別結構區分為不同的評分面向，並根據各題的實際要求，進一步訂定相對應的細項評分標準。

除了評分標準的設計外，語言模型單次推論所產生的結果亦可能存在變動。Tseng 等人提出 CodEv 自動程式評分架構，將題目內容、評分標準、學生程式碼及編譯結果整合為模型輸入內容，並由語言模型產生評分結果與文字回饋。該研究比較一般零樣本設定（Zero-shot）與加入思維鏈引導（Chain-of-Thought, CoT）的評分設定，並分別使用 Llama 3.1 (8B)、Gemma 2 (9B)、Llama 3.1 (70B) 及 GPT-4o，針對相同的學生作答進行多次獨立評分，再透過投票、平均數或中位數整合多次評分結果。實驗結果顯示，在結合思維鏈引導與多次評分後，Llama 3.1 (8B) 與 Gemma 2 (9B) 相較於人工評分的平均絕對誤差有所降低；而 Llama 3.1 (70B) 與 GPT-4o 並未呈現相同幅度的改善。此結果顯示，重複評分與結果整合主要有助於降低小規模模型的評分誤差，而大規模模型的單次評分結果已相對穩定，因此透過多次評分進行整合所帶來的效益較為有限[24]。然而，多次輸出具有一致性，不代表評分結果必然符合人工標準。Jukiewicz 比較多種語言模型對大量學生程式作業所產生的分數分布、評分變異，以及模型結果與教師評分之間的一致程度。研究結果顯示，不同模型對相同作業可能產生系統性的評分差異，例如模型評分往往較人類教師嚴格。而且模型彼此之間的評分一致程度可能高於教師評分之間的一致程度[16]。因此，模型輸出相近主要表

示其判斷趨勢相似，仍需透過人工評分或其他外部標準檢查其判定是否正確。

整體而言，現有研究已顯示語言模型可讀取學生原始碼，依題目特定評分標準產生分項判定與文字回饋，並可透過人工複核或多次推論整合評分結果。相關研究同時指出，評分結果會受到評分標準內容、模型選擇及推論方式影響，而模型間的一致程度亦不能取代與人工標準的比較。此外，若對每份學生作答進行多次完整評分，模型的推論次數與運算負擔將隨學生人數、題目數量及重複評分次數增加。若採用依輸入與輸出 Token 數量計費的雲端模型，其呼叫次數與推論成本亦會隨之累積。因此，將語言模型應用於程式自動評分時，除了評分準確性之外，亦須綜合考量模型規模、部署方式、推論資源及實際使用成本。

2-2 小型語言模型的界定、部署特性與訓練差異

前述研究顯示，語言模型可依題目、學生程式與評分規準進程式評分，但固定使用雲端大型語言模型進行多次完整推論，會隨評分規模增加模型呼叫量與使用費用。因此，近年研究開始關注參數規模較小、可於本地端或資源受限環境執行的語言模型，並探討其在特定任務中承擔推論工作的可行性。

現有研究對小型語言模型尚未形成統一的參數量界線。部分研究以模型參數量作為分類依據，亦有研究從記憶體需求、推論設備、執行速度與應用情境進行界定 [25]。Chen、Zhao 與 Han 整理大型與小型語言模型協作研究時，將小型語言模型描述為相較大型通用模型具有較低的運算與記憶體需求，並較適合部署於邊緣設備或資源受限環境；但在其整理的複雜推理與通用任務中，小型模型的整體結果通常仍低於大型模型 [9]。單以固定參數量作為區分標準，未必能完整反映模型的實際部署條件。相同參數規模的模型可能因模型架構、數值精度、上下文長度及推論框架不同，而具有不同的記憶體占用與執行速度；部

分參數量較高的模型經壓縮或量化後，亦可能在本地硬體上完成推論 [25]。因此，相關研究除模型參數量外，也會從計算需求、目標設備及應用情境討論小型語言模型。相較於依輸入與輸出 Token 計費的雲端大型語言模型，本地部署的小型模型不需針對每次推論支付 API 費用[8][25]，但仍需使用 GPU、記憶體、電力、模型儲存及系統維護等資源[5]。因此，兩者具有不同的成本結構：雲端模型的直接使用費用主要隨呼叫次數與 Token 數量增加，本地模型則主要消耗既有運算設備與維護資源，故本地推論不宜直接描述為零成本。

除了模型規模與部署條件外，小型語言模型的能力亦受到訓練資料組成、資料品質、合成資料比例、領域持續預訓練及後訓練方式影響。不同模型即使參數量相近，也可能因訓練目標不同，而在數學推理、程式生成、指令遵循及通用問答等任務上呈現不同結果。

因此，小型語言模型的差異不宜僅從參數量理解，亦應從其資料策略與訓練方式進行比較[2][12][13][15][26]。一種代表性策略是以經篩選的高品質資料與合成資料強化特定能力。Phi 系列即採取此一方向。早期 Phi 模型使用經篩選的教科書品質資料及由大型語言模型產生的合成程式教材，後續 Phi-4 則將高品質自然資料、合成資料、訓練課程及後訓練方法加以整合，以強化數學、程式及推理相關能力 [2][13]。此類策略的重點不是單純增加資料量，而是透過資料篩選與合成方式，提高訓練資料中與推理及問題解決相關的內容比例。相關實驗顯示，小型模型在特定推理基準上的結果，除參數量外，亦會受到資料品質與訓練流程設計影響。另一種策略是以大規模通用混合資料維持跨任務能力。Gemma 系列的訓練資料涵蓋自然語言文本、程式碼、數學及其他類型內容，並搭配大型模型的知識蒸餾（Knowledge Distillation）與後訓練方法，使模型能處理一般問答、摘要、推理及程式等相關任務[12]。此類模型並未只針對單一領域進行訓練，而是透過多類型資料與通用指令調整，在不同任務之間維持較廣

泛的適用範圍。相較於集中強化特定推理模式的資料策略，通用混合資料較重視跨領域任務的涵蓋程度。此外，部分小型模型採取通用預訓練後再以領域資料進行持續預訓練（Continual Pre-training）的方式，使模型能力集中於特定工作。Qwen 系列同時發展通用模型與程式領域模型，其中 Qwen2.5 通用模型使用大規模混合資料及多階段後訓練，以處理指令遵循、結構化資料、數學及程式任務[26]。在此基礎上，Qwen2.5-Coder 使用大量程式相關資料進行持續預訓練，並加入程式資料清理、合成資料生成及資料比例配置。研究結果顯示，Qwen2.5-Coder 在程式生成、補全、推理及修復等基準測試上，優於多個相近規模模型 [15]。此類策略顯示，模型可透過領域資料的持續訓練，使其能力更集中於程式相關任務。上述研究呈現三種不同的小型模型訓練方向，分別為高品質與合成資料強化、通用混合資料訓練，以及領域資料持續預訓練。這些差異說明，小型語言模型並非只是大型模型的縮小版本，其能力亦會隨資料來源、訓練目標與後訓練方式而改變。然而，數學推理、程式生成、程式修復或一般問答基準上的結果，不能直接等同於程式評分表現。模型在生成正確程式方面的結果較高，不代表其一定能正確判斷學生程式是否符合題目規範；模型具有較高的通用問答能力，也不代表其在語法與型別判斷上必然具有較高的一致程度。

因此，小型語言模型的選擇不宜只以參數量或單一整體排名作為依據，而應結合模型的訓練特性、部署條件及目標任務的實際結果進行判斷。當不同模型因資料與訓練方向而在各類任務上呈現差異時，如何將程式評分內容拆分為明確項目，並依各模型在不同評分內容上的結果進行配置，便成為後續結構化評分與多代理人任務分工所需探討的問題。

2-3 結構化評分與多代理人任務分工

程式作答通常同時涉及多種不同性質的判斷內容。例如，學生程式可能完成正確功能，但未使用題目指定的方法；也可能使用無誤的程式結構，卻無法正確處理輸入邊界；另有部分答案因語法、型別或方法呼叫錯誤而無法正常執行。若評分僅輸出單一總分，可能無法清楚呈現扣分原因，也難以判斷不同評分結果是否建立在相同的判定依據上。結構化評分準則會將整體表現拆分為多個可觀察項目，並分別描述各項目的判定條件。相較於直接給予整體印象分數，此類方式可記錄學生符合或未符合哪些要求，使評分者針對個別項目進行判定，也便於後續比較模型與人工評分在不同準則上的差異。對語言模型而言，明確列出評分項目與輸出要求，亦可界定模型需要檢查的內容，並使評分理由與分數具有可對照的依據。現有程式評分研究已採用多面向評分準則引導語言模型分析學生作答。Nagakalyani 等人提出 TA Buddy，將題目、學生程式與評分準則提供給 Code Llama，使模型依不同項目產生建議分數，再由助教接受或修改結果；研究結果顯示，該工具可縮短評分時間，且其建議分數與人工評分具有較高的一致程度 [19]。Cisneros-González 等人提出 JorGPT，以可編輯的評分準則引導多個大型語言模型分析學生程式，並保留教師複核機制 [10]。Tseng 等人在 CodEv 中，亦將程式評分區分為輸出正確性、程式可讀性與功能性，要求模型分別輸出各面向的分數與理由 [24]。上述研究顯示，透過明確的分項準則，語言模型可以產生可追溯至個別評分項目的判定結果，而不僅是單一總分 [10][19][24]。

除是否採用分項準則外，準則內容是否對應個別題目也會影響評分結果。Pathak 等人比較題目特定準則（Question-Specific Rubrics）與不依題目調整的通用準則（Question-Agnostic Rubrics），並以資料結構與演算法課程及物件導向程式設計課程的學生作答進行測試。研究結果顯示，題目特定準則在程式邏輯判

斷、回饋內容及與專家評分之對應程度上，優於通用準則[21]。此結果說明，固定的評分類別可作為上位架構，但實際判定項目仍應依個別題目的功能要求、指定限制及程式結構進行調整。結構化評分也有助於辨識總分相同但判定內容不同的情況。例如，一個評分結果可能判定程式功能錯誤但語法正確，另一個結果則可能判定功能正確但存在語法錯誤；兩者換算後可能得到相同總分，但其扣分依據與錯誤類型並不相同。因此，若研究只比較最終總分，可能無法辨識模型在個別準則上的判定差異。除總分外，同時保留各項準則的符合狀態、不同層面的分數與評分理由，較能呈現模型實際判斷的內容。

將評分拆分為多個項目，並不等同於完成任務分工。CodEv 雖採用多面向準則，但仍由同一模型完成所有判斷 [24]。當任務同時包含多種不同性質的分析工作時，多代理人系統（Multi-Agent Systems）則進一步透過角色與責任分配，使不同代理人分別處理部分子任務。此類系統不一定需要使用不同的基礎模型；即使採用相同模型，也可以透過不同的系統提示、輸入資訊、可用工具與輸出格式建立不同角色。多代理人分工的重點在於界定每個代理人的工作範圍，而不是單純增加模型數量。Hong 等人提出 MetaGPT，將軟體開發流程分配給產品經理、架構師、工程師與測試人員等代理人，並以標準化作業程序（SOP）規範不同角色之間的文件傳遞。實驗結果顯示，角色分工與結構化中間產物可用於完成軟體開發任務 [14]。Li 等人提出 CAMEL，透過角色設定與對話規則，使代理人依指定身分共同執行任務，顯示角色條件與互動規則可以改變代理人的合作方式與任務內容 [17]。雖然上述研究並非直接處理學生程式評分，但其方法說明，複雜工作可以先拆分為不同子任務，再為各角色設定相應的輸入、輸出與責任範圍。但多代理人分工也可能帶來新的問題。Cemri 等人分析了七種主流的多代理人語言模型框架及超過 1600 筆的執行軌跡（Traces），整理出 14 種失效模式，並將其歸納為系統設計失效（System Design Issues）、代理人之

間失配 (Inter-Agent Misalignment)，以及任務驗證失效 (Task Verification) 三類。研究結果顯示，角色說明不清、代理人未遵循指定角色、資訊共享不足及缺乏有效驗證，均可能使多代理人系統無法依原定流程完成任務[7]。此外，多代理人系統需要在代理人之間傳遞資訊並協調執行順序，當代理人及互動步驟增加時，也會提高系統的推論與協調負擔 [4]。因此，多代理人系統仍需建立清楚的角色責任、統一的資料格式，以及明確的結果驗證與整合方式。除了將評分內容拆解為多個項目，評分系統能否準確定位答案或判斷依據，同樣會影響最終判斷結果之可靠性。Zheng 與 Hsu (2026) 在評測 AI 代理人數據分析系統之輸出結果時發現，若不採用關鍵字輔助定位 (keyword-anchored) 僅依規則直接擷取答案，容易因輸出內容龐雜而定位錯誤；改採以關鍵字進行定位之抽取方式後，答案擷取之準確程度可獲得明顯提升[30]。此結果顯示，關鍵字作為輔助定位依據，前提是系統確實採用此一機制，方能有效降低評分系統因輸出內容複雜而產生的誤判風險。

2-4 大小模型協作、仲裁機制與成本考量

綜合上述研究，結構化評分準則可將程式評分要求轉換為具體且可逐項判定的內容，題目特定準則則有助於使評分項目更貼近個別作答要求；多代理人方法進一步提供將不同子任務配置至特定角色的架構基礎。不過，任務拆分與角色配置後，仍可能出現不同代理人判定不一致或結果難以整合的情況，因此尚需進一步探討如何檢查各角色輸出，以及何時引入其他模型處理有疑義的結果。

2-4-1 模型路由與串接式協作

前述研究顯示，不同規模與訓練方式的語言模型可能具有不同的任務表現，而結構化評分與多代理人分工則可將複雜任務拆分為較明確的子任務。然而，

若所有輸入均直接交由雲端大型語言模型處理，雖可利用其較強的推理能力，卻會大幅提高模型呼叫與推論成本；若完全由小型語言模型處理，則可能在面對邏輯較複雜或模型信心不足的邊緣案例（Edge Cases）時，產生較大的判斷誤差[8][11]。因此，大小語言模型協作的主要挑戰，在於如何依據任務複雜度與運算考量，進行有效的任務分配。

模型路由（Model Routing）是協調不同語言模型的一種方式，其核心是根據輸入特徵，選擇最適合處理該次請求的模型。Ong 等人提出 RouteLLM，利用人類偏好資料訓練路由器，使系統能在較強但成本較高的模型，與較弱但成本較低的模型之間進行動態選擇。研究結果顯示，在部分評估情境中，RouteLLM 可將使用成本降低至原來的一半以下，同時維持接近強大模型的回答品質 [20]。此研究說明，大小模型不必對所有輸入執行相同流程，而可透過路由機制在效能與成本之間進行動態調整。路由方法通常在推論開始前，依題目內容或輸入特徵選擇單一模型；而串接式協作（Cascading）則先由較小模型處理，再根據其輸出決定是否升級至大型模型。Chen 等人提出的 Data Shunt 以及 Zhang 等人提出的 COREA 框架皆屬此類作法 [8][28]。Data Shunt 透過評估小型模型的信心程度來決定分流方向，當信心低於特定門檻時，才將輸入資料轉交給大型模型處理，成功在亞馬遜商品情感分析等多項任務中，以不到原來三分之一的成本達到超越單一大型模型的準確率 [8]。類似地，COREA 同樣先由小型語言模型產生答案及信心分數，當信心不足時再交由大型模型接手。研究者另以強化學習方式校準小型模型的信心，使其信心分數更能反映回答正確的可能性。實驗結果顯示，相較於所有題目均使用大型語言模型，COREA 在領域外數學與非數學資料上分別降低了 21.5% 與 16.8% 的成本，而正確率的下降幅度則成功控制在兩個百分點以內。不過，以模型自行產生的信心作為升級依據，仍需處理「信心是否能正確反映答案可靠程度」的問題。之所以額外設計信心校準獎勵，

即是因為未經校準的自我信心不一定能直接作為穩定的路由訊號。此外，不同模型可能具有不同的信心表達方式，即使輸出相同的信心數值，其實際正確率也未必相同[28]。因此，條件式升級除了可使用單一模型的內部信心外，也可考慮以多次判定或不同判定結果之間的差異，作為需要進一步處理的觸發訊號。

2-4-2 分歧檢查與大模型仲裁

分歧檢查（Disagreement Checking）是另一種辨識潛在錯誤案例的方式。相較於要求模型直接評估自己的正確機率，分歧檢查會比較同一模型多次輸出，或不同模型對相同項目的判定是否一致。若多次判定相同，表示模型在該輸入下具有較穩定的輸出；若判定不同，則顯示該項目可能受到推論變動、評分準則理解差異或模型能力限制的影響。然而，分歧只能指出判定不一致，不能直接證明其中哪一個結果才是正確的，因此仍需要後續的選擇、驗證或仲裁機制。程式評分研究中也見到類似作法：透過固定的多次推論處理輸出變動，並以組內相關係數分析重複評分的一致程度 [24]。此方法能以多次輸出降低單次推論變動帶來的影響，但其重複推論與結果整合是固定執行於所有評分過程，而不是先檢查個別準則是否出現分歧，再只處理有疑義的部分。因此，固定重複評分與條件式分歧處理雖然都關注輸出穩定性，但前者會對所有案例增加相同的推論次數，後者則可將額外運算資源集中於已偵測到判定不一致的項目。當初步模型的判定出現分歧時，可由能力較強的語言模型擔任仲裁者（Arbitrator），根據原始輸入、評分準則及不同候選結果進行再次判斷。Zheng 等人探討以大型語言模型作為評審者（LLM-as-a-Judge），要求強大模型比較或評估其他模型的回答。研究結果顯示，GPT-4 等強大模型的判定與人工偏好具有較高的一致程度，說明大型語言模型可作為具擴充性的自動評估工具；但研究同時發現，模型評審仍可能受到答案位置、篇幅及自我偏好等因素影響 [29]。因此，仲裁

模型不應被視為絕對正確的標準，而應透過明確的評分準則、固定輸出格式與人工標準來持續評估其判定結果。仲裁與一般重新評分的差異，在於仲裁模型處理的是「已被辨識為有疑義」的案例，且其任務應集中於比較衝突判定並作出最終選擇。若仲裁提示僅要求大型模型重新完成所有評分，則原先小型模型的判斷與分歧資訊便未被充分利用，也可能增加不必要的 Token 消耗與呼叫成本。相較之下，將原始題目、學生答案、對應準則及衝突結果一併提供給仲裁模型，可使其針對具體爭議進行判斷，而不是重新處理所有已取得一致結論的項目。大小語言模型協作亦高度仰賴流程設計與結果驗證機制。正如前節所述，Cemri 等人的研究已指出，多模型協作系統若缺乏明確的角色邊界、資訊傳遞規範與驗證機制，極易導致流程失效。因此，大小模型協作除了需要決定何時升級，也需清楚界定各階段的輸入內容、判斷範圍及最終結果的整合方式[7]。

2-4-3 成本考量

王苙芸（2026）提出結合安全檢查代理與辯論式多代理協作之程式考試自動評分方法，由兩個大型語言模型（GPT-4o、Claude 3.5 Sonnet）獨立批改後，依核心判斷內容之語意相似度與分數差距門檻判斷是否達成共識，未達共識則進入代理人間之辯論回合，仍無法收斂時再由第三個大型語言模型（Gemini 2.5 Pro）進行仲裁。該研究並依仲裁結果回溯修正批改提詞，使評分規範隨批改次數逐步收斂。實證結果顯示，此一辯論式協作機制能提升批改說明之完整性與可解釋性，但批改與仲裁皆仰賴大型雲端語言模型執行，實際批改成本約為每份考卷 0.58 美元，研究者亦將「降低運算成本」列為未來亟待解決的研究方向[1]。

綜合上述研究，現有大語言模型協作方法主要可分為「輸入前路由」與「輸出後條件式升級」兩類。RouteLLM 根據輸入特徵預測適合的模型，

COREA 依小型語言模型的信心決定是否升級，而 CodEv 則透過固定的多次完整推論與結果整合處理評分變動。相關研究顯示，選擇性使用大型語言模型具有降低成本並維持結果品質的可能性；然而，以模型信心作為升級依據仍涉及信心校準問題，固定多次推論則會增加所有案例的推論量，且大型語言模型作為評審者時亦可能產生判定偏差。現有研究較少探討在結構化程式評分中，先針對個別評分項目檢查小型語言模型的判定分歧，再僅將具有疑義的項目交由大型語言模型仲裁，並同時比較其與人工評分的一致程度及推論成本。因此，此類條件式協作與定向仲裁機制仍有進一步研究與設計的空間。

三、研究方法

本研究提出一套結合小型語言模型分層評分與大型語言模型分歧仲裁之程式自動評分方法「小型語言模型異質分層與大型語言模型仲裁協作架構」，後續統一稱為「本論文架構」。

3-1 系統架構圖

本論文架構將學生作答區分為行為層、規範層與語言層，並由不同的小型語言模型分別負責各層初步評分。行為層主要判斷程式功能、邏輯與邊界處理；規範層主要判斷學生是否遵循題目指定要求；語言層則判斷 C# 語法、型別與基本程式結構是否正確。評分時，系統以三份分層結構化題目資料與學生答案作為輸入。原始題目文字僅用於前期建立結構化準則，不直接提供給評分模型。各評分層均進行 Round 1 與 Round 2 兩次獨立批改，系統再比較同一題、同一評分層的兩次結果，以判定是否存在分歧。若 Round 1 與 Round 2 的符合準則數相同，則視為該層評分結果一致；若符合準則數不同，則將該層標記為分歧，並交由大型語言模型重新判斷。大型語言模型僅處理實際發生分歧的層級，不重新評分其他已達成一致的層級。仲裁完成後，系統整合各層最終結果，並依題型所設定的行為層權重、語言層權重與規範係數計算各題分數，最後加總形成學生總分，如圖 1 所示。

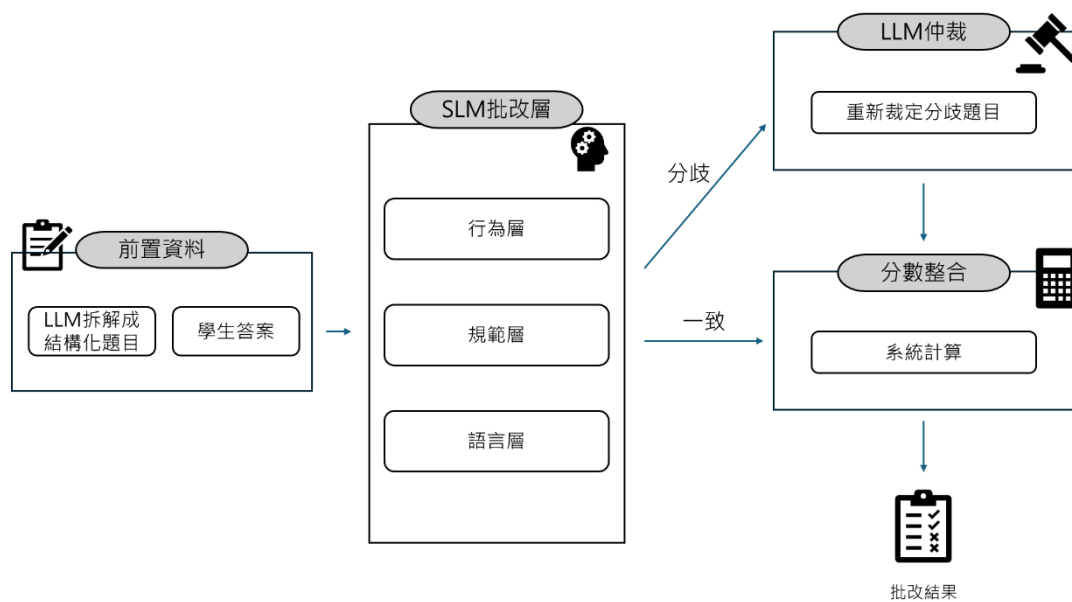


圖 1、系統架構圖 1

3-2 題目結構化與評分準則設計

相關研究顯示，不同語言模型及提示策略在學生程式評分上的表現可能存在差異，且提供評分範例有助於部分模型產生較接近教師評分的結果[22]。此外，語言模型在評估規範遵循等項目時，與教師評分之間仍可能存在較大差距[23]。本研究認為，程式作業評分時主要需關注三個不同面向：學生的邏輯思路是否正確、是否依照題目規定完成，以及所撰寫的程式在語法與運算結構上是否正確並能正常執行。這三個面向分別對應程式作業中常見的邏輯錯誤、未遵循題目規範與語法錯誤，其判斷重點彼此不同，不應相互混淆。因此，本研究依據上述三個面向，將評分內容區分為行為層、規範層與語言層，使每一層各自對應明確的評分重點，並降低不同面向之間重複判斷的情況，使評分結果能更清楚反映學生失分的具體原因。

3-2-1 三層評分準則定義

行為層（Behavior Layer）：行為層主要評估學生的解題邏輯與程式實際行

為。此層級旨在評估學生是否理解題目要求的運算過程，並能將其轉換為正確的步驟與判斷。即使程式碼寫法生疏、格式不夠工整，只要邏輯思路正確、能處理各種輸入情況，就代表學生掌握了這層要判斷的能力。因此，行為層主要判斷程式實際完成的功能、邏輯、演算法、控制流程及邊界條件是否正確，其核心問題為程式實際執行的行為（邏輯）是否符合題目要求。行為層判斷的是「解題思路本身是否成立」，而非「用哪一種具體寫法實現」。例如，若題目需要以迴圈逐步累加才能得出正確結果，學生卻完全省略迴圈、用單一運算式硬湊答案，或條件判斷不完整、未處理必要的邊界情況，都屬於行為層要判斷的邏輯缺陷。但學生究竟用 for、while 或 foreach 來實現這個迴圈，只要題目沒有特別指定，就不是行為層要管的問題——若題目明確要求特定迴圈類型，而學生迴圈邏輯正確但用錯了題目指定的迴圈類型，該問題由規範層的準則判定，行為層的準則中不會出現此類判斷項目。

規範層（Specification Layer）：程式設計課程除了考驗解題能力，往往也要求學生依照特定方式作答，例如使用指定的控制項、方法或變數命名。這類要求反映的是學生能否讀懂並遵循明確的作答規定，與程式功能是否正確是兩件事：學生可能寫出功能完全正確的程式，卻沒有依照題目指定的方式完成。因此，規範層主要判斷學生是否遵循題目指定的控制項、方法、變數、格式、命名、程式結構及限制條件，其核心問題為學生是否「照題目要求的方式」作答。例如，若題目明確要求必須使用 while 迴圈讀取檔案，並要求以指定名稱的控制項顯示內容，則學生即使改用其他迴圈完成相同的讀取與累加功能，或使用自行命名的控制項達成相同顯示效果，規範層仍依題目所列要求逐項判定不符合；但學生的讀取邏輯是否正確，仍由行為層獨立判斷，規範層的準則中不會出現功能正確性或語法正確性的判斷項目。反之，若題目未指定迴圈類型，僅要求輸入輸出格式，迴圈的選用即不列入規範層準則，學生採用任何寫法皆不受此

層限制。

語言層 (Syntax Layer)：程式邏輯再正確、作答方式再符合規定，若寫出來的程式碼本身有誤、無法通過編譯，仍不能算是完整的作答；語法與型別的正确性，是程式能否被實際執行的基本門檻。因此，語言層主要判斷 C# 程式碼在語法、型別、運算式、方法呼叫、變數宣告及基本程式結構上是否正確，其核心問題為程式是否具備正常執行的基本條件。延續前述檔案讀取的例子，語言層檢查的是 while 迴圈的括弧與大括號是否配對、StreamReader 物件的建立與 ReadLine() 的呼叫語法是否正確、控制項屬性的存取與累加運算的寫法是否正確。語言層的準則中不會出現邏輯正確性或指定寫法的判斷項目，且如果學生提交的是程式執行結果之文字描述，而非完整程式碼，語言層改以比對學生描述的輸出內容、順序、大小寫、標點及空格是否符合預期結果，彙整如表 1 所示。

表 1、三個評分層次之判斷定義

判斷層次	評分面向	核心判斷問題	判斷內容
行為層 (Behavior)	語意與 邏輯	程式「實際做了什麼」是否正確	功能是否完整、邏輯是否正確、是否處理邊界條件
規範層 (Specification)	規則 指令遵循	是否「照題目要求的方式」撰寫	是否使用指定控制項、格式，是否違反限制條件
語言層 (Syntax)	語法與 運算結構	程式是否「能正常執行」	是否存在語法錯誤、型別錯誤或基本結構缺失

透過上述分層，本論文架構將功能與邏輯正確性、題目要求遵循程度，以及程式語言合法性分別處理，使各模型僅針對其負責層級進行判斷。

3-2-2 題型分類與權重設計

本研究的評分設計以題目主要評量內容為依據，將題目分為規則題、邏輯題及問答題三種題型。題型用於決定行為層與語言層在最終計分中的權重，兩項權重總和固定為 1。規範層則是依符合比例形成規範係數，作用於行為層與語言層的加權結果。此種設計用以反映題目明確指定之必要要求具有優先約束作用：當學生完整符合題目要求時，規範係數為 1；當學生僅符合部分規範時，最終分數依其符合比例調整；若完全未遵循題目所列的必要要求，規範係數為 0，使該題加權結果歸零。規範係數為 $R = \frac{\text{規範層符合數}}{\text{規範層總準則數}}$ ，假設 Q8 的規範層總準則數為 2，規範層符合準則數為 1， $R = \frac{1}{2}$ 。

規則題主要評量學生是否依題目指定方式完成程式功能。此類題目同時重視程式實際行為與程式語言結構，但實際行為之重要性仍高於語言結構，因此行為層權重設定為 0.6，語言層權重設定為 0.4，規範層依符合比例作為乘法係數。邏輯題主要評量學生的程式邏輯、演算法、條件判斷、迴圈處理及邊界情況。此類題目的核心在於解題流程與程式行為是否正確，因此行為層權重設定為 0.7，語言層權重設定為 0.3，規範層同樣依符合比例作為乘法係數。問答題主要評量學生對 C# 程式執行結果與語言概念的理解。學生不需撰寫完整程式，而是依據題目提供的程式內容回答預期執行結果，因此行為層權重設定為 0，語言層權重設定為 1，規範係數固定為 1；其行為層與規範層準則數皆設定為 0，彙整如表 2 所示，相關題型與題目範例收錄於附錄 1 之表 7。

表 1、題型分類權重配置

題型	行為層權重 (w_b)	語言層權重 (w_s)	規範係數(R)
規則題	0.6	0.4	依符合比例計算
邏輯題	0.7	0.3	依符合比例計算
問答題	0	1	R 固定為 1

上述權重設定以題型作為分類單位，同一題型下之各題共用相同權重組合，未再依個別題目的難易程度或準則數量進行調整。雖然權重亦可進一步依各題特性進行更細緻的設定，但本研究主要探討不同題型下固定權重配置之實驗結果，因此目前採用依題型分類的固定權重進行實驗。

3-2-3 關鍵字設計

在輸出內容較為複雜的情況下，以關鍵字作為輔助定位與參照依據，有助於降低答案擷取或判斷時產生誤判的情況 [30]。在結構化題目檔中，除逐項評分準則外，本研究另為各題保存「關鍵字 (Keywords)」作為模型判斷時的輔助參照。部分題目包含判斷作答正確與否所需的具體資訊，若將此類細節全部納入評分準則，容易使準則內容過於冗長；若完全交由模型自行推導，則可能產生不一致的判斷結果。因此，本研究將部分必要且具體的參照資訊自評分準則中獨立整理，並以關鍵字形式提供給模型。關鍵字內容依題目性質大致可分為三種類型：其一為與題目核心概念相關的技術詞彙；其二為題目敘述未完整列舉，但屬於判斷正確與否所需的完整正確值集合；其三為針對輸出精確性要求較高之題目所提供的預期輸出值全文。以星座判定題為例，題目敘述可能僅

以「Aries、Taurus、Gemini 等」表示十二個星座名稱，而未逐一列出完整內容，因此本研究於關鍵字欄位中保存十二個星座的完整英文名稱供模型參照，其功能僅作為輔助性的比對參照，使模型在依循逐項準則進行判斷時，能取得較一致且完整的必要資訊，相關結構化題目範例收錄於附錄 2 之表 8。

3-3 模型選擇與設計和仲裁機制

本論文架構依三層評分任務的性質，分別配置具有不同能力特性的小型語言模型；大型語言模型則僅於分歧時介入仲裁。本節先說明模型角色配置，再說明各層提詞設計的原則，最後說明大型語言模型仲裁角色與仲裁提詞的內容。

3-3-1 小型語言模型角色配置

本論文架構依三個評分層級的任務性質，配置具不同訓練特性與能力取向的小型語言模型。模型選擇主要考量模型的訓練方向、各評分任務所需處理的資訊類型、本地部署可行性，以及實際推論成本，使模型特性與評分內容盡可能相互配合。三個小型語言模型皆為可透過 Ollama 量化部署於單張消費級 GPU 的開放權重模型，主要負責大量且重複的初步評分工作。本研究依第二章整理之模型訓練特性與任務能力進行角色配置，但模型於程式生成、數學推理或通用問答基準中的表現，不能直接等同於其程式評分能力。因此，本節所提出的配置主要是根據模型訓練方向與各評分層需求所建立的理論適配關係，實際適用性仍須透過後續實驗加以檢驗。

行為層由 Qwen2.5-Coder:7B 負責。Hui 等人（2024）提出 Qwen2.5-Coder，該模型以 Qwen2.5 為基礎，並使用約 5.5 兆個 Token 的程式相關資料進行持續預訓練，其資料涵蓋原始碼、程式與自然語言對齊資料，以及合成程式資料。研究者透過程式生成、程式補全、程式推理與程式修復等基準進行評估，結果顯

示，Qwen2.5-Coder 在多項程式任務上優於若干相近參數規模的模型[15]。上述研究結果主要反映 Qwen2.5-Coder 在程式碼理解與處理任務上的能力，尚不能直接證明其在學生程式評分中的表現。本研究之行為層主要分析程式實際完成的功能、演算法邏輯、控制流程與邊界條件，需要進一步推演程式於不同輸入下的執行行為。由於此類任務與 Qwen2.5-Coder 所強化的程式推理及修復能力具有較高的理論適配性，因此本研究將其配置於行為層，專門處理程式功能與邏輯相關判斷，而不負責題目規範或語法合法性的評估。

規範層由 Gemma 4 E4B 負責。Gemma Team 等人（2024）說明，Gemma 系列使用涵蓋自然語言文本、程式碼與數學內容的混合資料進行訓練，並結合知識蒸餾與後訓練方法，以提升模型在通用語言理解、推理與指令遵循任務中的能力。研究結果顯示，Gemma 2 在多項通用語言、推理與程式基準中，能以相對有限的參數規模維持一定的任務表現[12]。上述研究顯示，Gemma 系列的訓練方向較偏向跨任務的通用資訊處理，而非僅集中於單一程式領域。本研究之規範層主要檢查學生是否依題目要求使用指定控制項、方法、變數名稱、格式與程式結構，核心工作是理解題目指令，並將學生答案與各項規範逐一核對。因此，本研究選用可於研究設備上部署的 Gemma 4 E4B 執行規範層評分。此項配置主要基於 Gemma 系列的通用訓練方向、指令處理能力與本地部署條件，並非主張既有文獻已直接證明 Gemma 4 E4B 為規範層的最佳評分模型。

語言層由 Phi-4 負責。Abdin 等人（2024）提出 Phi-4，並在訓練過程中結合高品質自然資料、合成資料及多階段後訓練，以強化模型在數學推理、問題解決與程式相關任務中的能力。研究者亦於後訓練階段採用依評分準則引導的自我修正及生成結果評估方法，結果顯示 Phi-4 在多項推理與知識基準中具有良好表現[2]。Phi-4 技術報告所呈現的主要是模型的整體推理能力與準則導向訓練特性，並未直接驗證其對 C# 學生程式進行語法評分的能力。本研究之語言層採用

明確且逐項的結構化準則，主要檢查 C# 程式碼的語法、型別、運算式、方法呼叫、變數宣告及基本程式結構；對問答形式的問答題，則比對學生描述的輸出內容、順序與格式。由於此類任務需要依固定規則進行結構化判斷，本研究推論 Phi-4 的推理能力與準則導向後訓練方式，與語言層的任務形式具有一定適配性，因此將其配置於語言層，實際效果則由後續實驗進行檢驗。

綜合而言，本論文架構將程式功能與演算法推理交由程式領域特化的 Qwen2.5-Coder 處理，將題目規範與指令遵循交由通用型 Gemma 4 E4B 處理，並將語法、型別及結構化輸出比對交由 Phi-4 處理。此種配置的核心不在於宣稱個別模型於特定評分層級具有絕對優勢，而在於透過任務拆解與異質模型配置，檢驗不同訓練取向之小型語言模型能否分別承擔性質不同的評分工作，並在大型語言模型分歧仲裁機制的支援下，兼顧評分結果與推論成本，彙整如表 3 所示。

表 2、小型語言模型之角色配置

代理人	層級	模型	參數規模
SLM1	行為層	Qwen2.5-Coder	約 7B
SLM2	規範層	Gemma 4 E4B	約 8B
SLM3	語言層	Phi-4	約 14B

3-3-2 小型語言模型提詞設計

本研究依行為層、規範層與語言層分別建立評分提示詞。各層提示詞包含評分層定義、核心判斷問題、判斷內容、跨層限制、結構化題目資料、學生答案及指定輸出格式，使小型語言模型只針對其負責的評分面向進行判斷。行為

層提示詞要求模型只判斷程式實際功能、邏輯及邊界處理，不考慮題目指定寫法，也不直接因語法、型別或編譯問題扣分。規範層提示詞要求模型只判斷學生是否遵循題目要求的控制項、格式、方法及限制，不因功能或語法錯誤重複扣分。語言層提示詞則要求模型只判斷語法、型別、運算結構與基本程式結構，不因功能邏輯錯誤或未採用指定方式扣分。

為降低模型直接產生結論時遺漏部分準則的可能性，本研究在三個小型語言模型的輸出格式中設置「推理過程（analysis）」欄位，要求模型先逐項比對準則與學生答案，再產生各項準則判定及分數。此種先分析、後判定的方式，可協助模型整理準則、學生答案與評分結果之間的關係，並保留後續分歧分析與仲裁可使用的判斷紀錄。各項準則採二元方式判定，完全符合者記為 1；部分符合、不符合，或無法由學生答案證明者記為 0。三個模型皆依準則記錄每一項判定結果，分數則以「符合準則數／準則總數」表示。提示詞要求模型僅輸出合法 JSON，不得加入 Markdown 標籤、前言或其他文字，各語言模型提詞請詳閱附錄 3。

3-3-3 大型語言模型仲裁選擇設計與仲裁流程

本研究選用 Claude Sonnet 5 作為大型語言模型仲裁者。儘管 Claude Sonnet 5 並非現今單次 API 呼叫費用最低的大型語言模型，但若綜合考量其在結構化指令遵循、複雜程式碼理解、判斷內容的穩定性，以及處理輸出格式修正與重試需求等面向，其能力特性符合本研究仲裁任務的需求。根據 Anthropic 發布之 Claude Sonnet 5 System Card，該模型為 Claude Sonnet 4.6 的後續版本，並在程式碼處理、代理人任務及多項專業工作評估中呈現能力提升[3]。考量本研究之仲裁工作需要處理學生程式碼、結構化評分準則及不同評分結果，本研究將 Claude Sonnet 5 配置為大型語言模型仲裁者。

仲裁模型之角色僅限於：當同一評分層的 Round 1 與 Round 2 符合準則數不同時，針對實際發生得分分歧的層級重新進行逐項判斷。系統於分歧判定階段逐層比對各題之 Round 1 與 Round 2 結果；若某一層級的符合準則數不同，則將該層級記錄為分歧層，並保留該層 Round 1 與 Round 2 的分數及分析內容；若符合準則數一致，則不將該層納入仲裁。進入仲裁階段後，系統以題目為單位呼叫一次大型語言模型，並將同一題中所有發生分歧的層級整合於同一次仲裁請求中。仲裁請求包含該題學生答案一份、各分歧層之評分準則、各分歧層 Round 1 與 Round 2 的分數及推理過程，以及三層判斷界線的固定說明。未發生分歧之層級，其評分準則與 Round 1、Round 2 結果則不納入仲裁請求。系統要求大型語言模型於同一次回覆中，分別針對各分歧層輸出獨立的分數與判斷理由，且各層仍須依其既定判斷範圍分別處理。大型語言模型不得直接選擇或採納 Round 1 或 Round 2 的結果，而須重新依據官方評分準則與學生答案進行逐項判斷。Round 1 與 Round 2 的分數與分析內容僅作為仲裁模型定位爭議來源之參考，不作為直接採納的最終答案。每項準則仍採二元方式判定：完全符合者記為 1；部分符合、不符合，或無法由學生答案證明者記為 0。仲裁模型僅處理實際發生分歧的層級，不重新評分已達成得分一致的其他層級。若同一題中有兩個以上層級發生分歧，則於同一次仲裁請求中一併處理，並分別輸出各分歧層之分數與總結理由。本研究分歧判定以整個評分層級的結果作為最小判定單位，而非逐一比較各項準則。若以單一準則作為分歧判定單位，並將各項分歧分別送交大型語言模型進行仲裁，當同一題出現多項準則分歧時，將增加大型語言模型的呼叫次數及運算成本，與本研究降低大型語言模型使用成本之目的不符。因此，本研究選擇以「層級」作為分歧判定的最小單位。

3-4 結果整合與計分方式

批改與仲裁完成後，系統依各層是否出現得分分歧，決定最終採用的評分

來源：得分一致層採用 SLM 完整結果，發生分歧的層採用大型語言模型仲裁結果分數整合與計算均由程式完成。

計算公式會使用行為層比率 $B = \frac{\text{行為層符合數}}{\text{行為層總準則數}}$ 和語言層比率 $S = \frac{\text{語言層符合數}}{\text{語言層總準則數}}$

各自乘上權重並相加，再乘上規範係數 $R = \frac{\text{規範層符合數}}{\text{規範層總準則數}}$ 和該題配分 P ，而權重會

因為題目的類型不同，行為權重(w_b)、語言權重(w_s)有所變動，規則題($w_b = 0.6$ ， $w_s = 0.4$)，邏輯題($w_b = 0.7$ ， $w_s = 0.3$)，而問答題較為特殊，因該題的行為層和規範層是不做判斷，只有語言層會進行判斷，所以行為層權重為 0，語言權重為 1，規範係數固定為 1，($w_b = 0$ ， $w_s = 1$ ， $R = 1$)，具體公式如下：

$$Score = [(B \times w_b) + (S \times w_s)] \times R \times P \quad \text{式(1)}$$

以規則題為例($w_b = 0.6$ 、 $w_s = 0.4$ 、 $P = 10$)，若行為層符合準則數 4，總準則數為 4，行為層比率為 $B = \frac{4}{4}$ 。語言層符合準則數 3，總準則數為 6，比率為 $S = \frac{3}{6}$ 。規範層符合準則數 1，總準則數為 2，規範係數 $R = \frac{1}{2}$ ，則該題得分 4，計算如下：

$$\left[\left(\frac{4}{4} \times 0.6\right) + \left(\frac{3}{6} \times 0.4\right)\right] \times \frac{1}{2} \times 10 = 4$$

問答題為例($w_b = 0$ 、 $w_s = 1$ 、 $P = 10$)，行為層總準則數為 0，行為層比率為 $B = 0$ 。語言層符合準則數 2，總準則數為 4，比率為 $S = \frac{2}{4}$ 。規範係數固定為 $R = 1$ ，則該題得分 5，計算如下：

$$[(0 \times 0) + \left(\frac{2}{4} \times 1\right)] \times 1 \times 10 = 5$$

四、研究結果與分析

本章旨在透過實驗數據，檢驗本論文提出之「小型語言模型異質分層與大型語言模型仲裁協作架構」是否能達成降低成本之研究目標。具體而言，本章將探討在不同系統配置下，評分表現與推論成本各自的變化趨勢，藉此確認本論文架構是否真能在維持高水準評分品質的前提下，有效且大幅地降低大型語言模型之使用成本。測試資料採用三份對應不同成績區間（低分、中等分、高分）之學生考卷（分別標記為 C60、C80、C100）。每份考卷皆包含 10 題，題目內容涵蓋大學程式設計課程之教材習題，以及 ZeroJudge 線上程式設計評測平台之題庫[27]。為建立客觀之比較基準，三份考卷皆由人工依據三層結構化評分標準進行批改，並據此建立人工標準分數（C60 為 14.58 分、C80 為 77.92 分、C100 為 100.00 分）。為逐步驗證本論文架構之各項設計效益，本研究設計了以下四組比較模型，各組均對三份考卷重複批改六次：

1. 對照組 B（LLM 單一模型）：全部評分工作由單一大型語言模型（Claude Sonnet 5）執行，並由其自我仲裁分歧。
2. 對照組 C（同質 SLM 單一模型）：三層評分工作皆由單一小型語言模型（Gemma 4 E4B）於同一次呼叫中處理，並自我仲裁分歧。
3. 對照組 D（同質 SLM 分層 + LLM 仲裁）：任務拆解為三層，但三層皆由（Gemma 4 E4B）分別獨立執行，分歧時交由 LLM 仲裁。
4. 實驗組 V4（異質 SLM 分層 + LLM 仲裁，本論文架構）：異質 SLM 分工（行為層 Qwen2.5-Coder、規範層 Gemma 4 E4B、語言層 Phi-4），並交由 LLM 進行分歧仲裁。

為建立評分品質之參照基準，本研究首先觀察在結構化批改下，大型語言模型（對照組 B）與人工批改標準之落差，並以平均絕對偏差（Mean Absolute Deviation, MAD）檢視模型於重複批改時的分數一致性，其公式如下：

$$MAD = \frac{1}{n} \sum_{i=1}^n |x_i - \bar{x}| \quad \text{式(2)}$$

其中， n 為重複批改次數， x_i 代表同一份考卷進行第 i 次批改所得之分數， $\bar{x}$ 為對同一份考卷進行 n 次批改分數之平均值。MAD 數值越小，代表重複批改同一份考卷時的給分越穩定，但 MAD 僅反映重複批改結果的集中程度，不代表平均評分結果必然較接近人工標準。

4-1 結構化評分下單一模型配置之表現

透過明確的結構化分項評分準則，語言模型能產生可追溯至個別評分項目的判定結果[24]，且針對題目設計的特定準則（Question-Specific Rubrics），相較於通用準則較能對應人類專家的評分結果[21]。本研究依此設計了三層結構化評分準則，所有實驗組與對照組均在此結構化評分之基礎上執行。因此，本章之比較起點，即為大型語言模型在結構化評分下之實際表現。

表 3、對照組 B 與對照組 C 之結果比較

成績區間		C60	C80	C100
人工批改標準		14.58	77.92	100.00
對照組 B	平均	15.89	72.80	100.00
	MAD	1.10	0.45	0.00
	與人工標準差異	1.31	-5.12	0.00
對照組 C	平均	19.11	77.50	91.33
	MAD	0.15	0.67	0.00
	與人工標準差異	4.53	-0.42	-8.67

如表 4 所示，對照組 B 與人工批改標準之差距介於-5.12 至 1.31 分之間。C60 中，對照組 B 較人工標準高 1.31 分、C80 中低 5.12 分、C100 則與人工標準完全一致。此結果顯示，即使採用能力較強的大型語言模型，在結構化評分設定下，其結果亦不一定與人工批改完全相同。本研究主要探討小型語言模型能否維持接近大型語言模型之評分水準，因此後續將各組與對照組 B 之分數差異納入比較。

在此基準上，本研究設計對照組 C，以檢驗最直接的小型語言模型替換方式。對照組 C 採用 Gemma 4 E4B 作為單一評分模型，將對照組 B 所使用之大型語言模型替換為單一小型語言模型，於同一次呼叫中一併處理三層評分，並於分歧時自我仲裁，其餘實驗條件維持一致。Gemma 4 E4B 並非本研究選用模型中參數規模最高者，但其通用型定位較符合同時處理三種異質評分任務的需求，因此作為「單一模型承擔全部評分工作」的代表模型。如表 4 所示，就平均分

數而言，對照組 C 在 C80 的平均分數為 77.50 分，與人工批改標準僅相差 0.42 分，優於對照組 B 的 5.12 分差距。然而，在 C60 與 C100 中，對照組 C 分別與人工標準相差 4.53 分及 8.67 分，顯示單一小型語言模型在不同成績區間中的評分表現並不一致，尤其在高分考卷中出現較明顯的低估情形。就重複批改穩定度而言，對照組 C 在 C60、C80 與 C100 的 MAD 分別為 0.15、0.67 及 0，整體波動幅度較小。其中，C100 之 MAD 為 0，表示多次批改結果完全一致；但其平均分數僅為 91.33 分，與人工標準及對照組 B 均相差 8.67 分。此結果顯示，較低的 MAD 僅代表模型能穩定地產生相近分數，並不代表其評分判斷必然正確。換言之，模型亦可能穩定地產生相同方向的評分偏誤。

此外，本研究在實際輸出中觀察到，對照組 C 於同一次呼叫中處理三層評分時，偶爾出現跨層判斷混淆與虛構不存在評分標準的情形。例如，在批改問答題時，依本研究的三層定義，該題型僅於語言層設有評分標準，行為層與規範層之評分項目均為空值；然而，Gemma 4 E4B 仍可能自行產生行為層與規範層的判斷內容。此現象顯示，在同一次呼叫中同時輸入三層定義與評分標準，可能提高模型混淆不同層級判斷範圍的風險。上述跨層錯亂未直接影響該問答題的最終分數，原因在於本研究計分公式中，問答題的行為層權重固定為 0，規範層係數固定為 1，最終分數僅依語言層符合比率計算。因此，即使模型錯誤產生行為層或規範層判斷，其結果亦不會進入最終計分。然而，這類輸出仍反映單一小型語言模型在同一次呼叫中同時承擔多重異質評分角色時，存在評分邊界混淆與跨層污染的風險。

綜合而言，對照組 C 在部分成績區間能產生接近人工標準的平均分數，且其 MAD 整體較低，顯示 Gemma 4 E4B 具有一定的重複批改穩定性。然而，其在 C100 中呈現「低波動但高偏差」的結果，且實際輸出中仍出現跨層判斷混淆。由此可知，單一小型語言模型雖可能穩定地完成多層評分，但穩定性不必然等

同於評分正確性，亦不能完全避免多重評分角色集中於同一次呼叫所產生的跨層干擾。此結果構成下一節同時檢驗任務拆解與模型異質化配置能否改善跨層干擾與評分表現之實驗基礎。

4-2 任務拆解與模型異質化之比較

依據 4-1 之觀察，對照組 C 在同一次呼叫中集中處理三層評分時，存在跨層判斷混淆的風險。為檢驗評分表現能否透過任務拆解與模型配置獲得改善，本研究進一步設計對照組 D 與實驗組 V4：對照組 D 將行為層、規範層與語言層拆分為三個獨立評分任務，但三層仍統一由 Gemma 4 E4B 執行；實驗組 V4 同樣將評分拆分為三個獨立任務，但依各層任務性質配置不同的小型語言模型——行為層配置經程式領域持續預訓練的 Qwen2.5-Coder，負責判斷程式功能、演算法與邊界條件；規範層配置通用型 Gemma 4 E4B，負責逐項比對學生是否使用指定控制項、方法與格式；語言層配置 Phi-4，負責判斷 C# 語法、型別與方法呼叫是否合法。三組之評分任務皆依三層架構進行判斷，差異在於分歧發生時的仲裁方式與模型配置：對照組 C 由同一模型自我仲裁；對照組 D 與實驗組 V4 則於分歧時交由大型語言模型仲裁，兩者之差異僅在於拆分後各層是否配置不同模型。本節即以此設計，同時比較對照組 C、對照組 D 與實驗組 V4 之表現，彙整如表 5 所示。

表 4、對照組 C 與對照組 D 與實驗組 V4 之結果比較

成績區間		C60	C80	C100
對照組 B 平均		15.89	72.8	100.00
對照組 C	平均	19.11	77.50	91.33
	MAD	0.15	0.67	0.00
	與對照組 B 差異	3.22	4.70	-8.67
對照組 D	平均	16.85	76.25	88.46
	MAD	1.83	0.83	5.42
	與對照組 B 差異	0.96	3.45	-11.54
實驗組 V4	平均	14.93	72.46	97.17
	MAD	0.96	1.72	1.56
	與對照組 B 差異	-0.96	-0.34	-2.83

就各組與對照組 B 的平均分數差異而言，對照組 C 在 C60 中高出 3.22 分、C80 中高出 4.70 分、C100 中則低 8.67 分，三個成績區間整體平均相差約 5.53 分。對照組 D 在 C60 中高出 0.96 分、C80 中高出 3.45 分、C100 中則低 11.54 分，三個成績區間整體平均相差約 5.32 分，與對照組 C 相近。此結果顯示，單純將三層評分由同一次呼叫拆分為獨立處理，尚未使整體評分結果更接近對照組 B。實驗組 V4 在 C60、C80 與 C100 中則分別低 0.96 分、0.34 分及 2.83 分，三個成績區間整體平均相差約 1.38 分，低於對照組 C 與對照組 D，顯示依各層任務特性配置不同模型後，評分結果較為接近對照組 B。

在重複批改穩定度 (MAD) 上，對照組 C 在 C60、C80 與 C100 中的 MAD 分別為 0.15、0.67 及 0.00。其數值雖低，但 C100 的評分結果與對照組 B 相差 8.67 分，顯示較低的批改波動不一定代表評分結果接近對照組 B，也可能表示模型在多次批改中穩定地產生具有相同方向偏差的結果。對照組 D 的 MAD 分別為 1.83、0.83 及 5.42，三個成績區間皆高於對照組 C，顯示將同一模型拆分為三個獨立任務後，重複批改的波動並未降低。實驗組 V4 的 MAD 分別為 0.96、1.72 及 1.56；除 C80 外，其於 C60 與 C100 的 MAD 均低於對照組 D，同時三個成績區間與對照組 B 的整體平均分數差距亦為三組中最低。

綜合以上結果可知，對照組 C 與對照組 D 在三個成績區間中，與對照組 B 整體平均相差約 5.5 分，分別為 5.53 分與 5.32 分。此結果顯示，在本研究設定下，單純將三層評分拆分為獨立任務，尚未使整體評分結果明顯更接近對照組 B，且部分成績區間的重複批改波動反而上升。進一步採用異質模型配置後，實驗組 V4 與對照組 B 整體平均相差約 1.38 分，且其 C60 與 C100 的 MAD 均低於對照組 D；但 V4 在 C80 的 MAD 仍高於對照組 D，因此不能概括認為所有成績區間的重複批改穩定度皆獲得改善。在批改時間方面，對照組 D 的三層評分皆由 Gemma 4 E4B 依序執行；實驗組 V4 則由不同模型平行處理三層任務，其完成考卷批改所需時間約為對照組 D 的一半。整體而言，本研究結果顯示，異質模型配置可能有助於縮小與對照組 B 的評分差距並縮短執行時間，但其表現仍會受到模型組合、執行方式與成績區間影響。

4-3 整體降低成本之驗證

前述比較顯示，實驗組 V4 相較於同質分層配置，在評分接近程度與執行效率上具有較佳表現。為進一步確認本論文架構是否達成降低大型語言模型使用成本之研究目標，本節進一步比較實驗組 V4 與大型語言模型單一配置之對照組

B，觀察 V4 能否在維持接近大型語言模型評分結果的同時，大幅降低大型語言模型之 Token 使用量。對照組 B 的全部評分與分歧仲裁工作皆由大型語言模型執行；實驗組 V4 則先由三個小型語言模型分別完成行為層、規範層與語言層評分，僅在兩次評分結果出現分歧時，才交由大型語言模型仲裁。因此，本節主要比較兩組的最終平均分數、重複批改穩定度，以及大型語言模型 Token 使用量，彙整如表 6 所示。

表 6、對照組 B 與實驗組 V4 之比較

成績區間		C60	C80	C100
對照組 B	平均	15.89	72.80	100.00
	MAD	1.10	0.45	0.00
	B 總使用 Token 數量	73877	68840	59259
實驗組 V4	平均	14.93	72.46	97.17
	MAD	0.96	1.72	1.56
	與對照組 B 差距	-0.96	-0.34	-2.83
	V4 總使用 Token 數量	9304	12522	5899

實驗組 V4 於三種成績區間中與對照組 B 之平均差距皆控制在 3 分以內。本節進一步檢視 V4 是否以更低的大型語言模型使用成本，維持此一評分水準。在重複批改穩定度方面，對照組 B 於 C60、C80 及 C100 的 MAD 分別為 1.10、0.45 及 0.00，實驗組 V4 則分別為 0.96、1.72 及 1.56。V4 在 C60 中的 MAD 略低於對照組 B，表示其低分考卷之重複批改波動較小；但在 C80 與 C100 中，

V4 的 MAD 高於對照組 B，顯示大型語言模型單一配置在中分與高分考卷中的重複批改結果較為集中。由此可知，V4 雖能維持接近對照組 B 的平均分數，但其重複批改穩定度尚未在所有成績區間中達到大型語言模型之水準。在大型語言模型 Token 使用量方面，對照組 B 於 C60、C80 及 C100 分別使用 73,877、68,840 及 59,259 個 Token；實驗組 V4 則分別使用 9,304、12,522 及 5,899 個 Token。V4 於三種成績區間中的大型語言模型 Token 使用量分別減少約 87.4%、81.8% 及 90.0%，顯示其僅在分歧項目需要仲裁時呼叫大型語言模型，能大幅減少大型語言模型參與主要評分流程的比例。若以三種成績區間合計，對照組 B 共使用 201,976 個大型語言模型 Token，實驗組 V4 共使用 27,725 個 Token，總計減少 174,251 個 Token。V4 僅使用約 13.7% 的大型語言模型 Token，即可將三種成績區間中的最終平均分數控制在與對照組 B 相差 3 分以內。

綜合而言，實驗組 V4 並非在所有指標上皆優於對照組 B。對照組 B 於 C80 與 C100 中具有較低的 MAD，顯示其重複批改穩定度仍具有優勢；然而，V4 在三種成績區間中的平均分數皆維持接近對照組 B 的水準，同時將大型語言模型 Token 使用量降低 81.8% 至 90.0%，整體減少約 86.3%。此結果顯示，本論文架構所達成的降低成本，並非單純追求最低 Token 用量或完全取代大型語言模型，而是在保留大型語言模型作為分歧仲裁機制的前提下，將主要評分工作交由小型語言模型執行，使系統在本研究樣本中以較低的大型語言模型 Token 使用量，維持接近大型語言模型單一配置之評分結果。因此，本研究結果支持小型語言模型異質分層與大型語言模型仲裁協作架構具備實際應用價值，並顯示 V4 已達成本研究所設定之降低成本目標。

4-4 本章小結

本章透過對照組 B、對照組 C、對照組 D 與實驗組 V4 之比較，逐步檢驗單

一小型語言模型替代、任務拆解、模型異質化及選擇性大型語言模型仲裁等設計，對評分結果、重複批改穩定度、執行時間與大型語言模型使用成本所產生的影響。

首先，對照組 B 與對照組 C 之比較結果顯示，單一 Gemma 4 E4B 雖能在部分成績區間產生接近大型語言模型或人工批改標準的平均分數，且具有一定的重複批改穩定度，但其在 C100 中呈現低 MAD 卻具有明顯分數偏差的情形。此外，對照組 C 於同一次呼叫中同時處理三層評分時，亦可能出現跨層判斷混淆與虛構不存在評分標準等問題。此結果顯示，穩定的重複批改結果不必然代表評分結果較為正確，單一小型語言模型同時承擔多種異質評分任務亦可能產生固定方向的偏誤。其次，對照組 C 與對照組 D 之比較結果顯示，將三層評分內容拆分為獨立任務後，對照組 D 在 C60 與 C80 中縮小了與對照組 B 之間的分數差距，但在 C100 中的差距反而擴大，且三種成績區間的 MAD 皆高於對照組 C。由此可知，任務拆解雖可降低多層內容集中於同一次提示詞所造成的跨層干擾風險，但若三層仍由同一小型語言模型執行，尚不足以在不同成績區間中一致改善評分接近程度與重複批改穩定度。進一步比較對照組 D 與實驗組 V4 後發現，在相同的三層任務拆解與大型語言模型仲裁機制下，依各層任務特性配置不同小型語言模型，可改善同質模型分層配置之限制。實驗組 V4 在 C80 與 C100 中的平均分數均較對照組 D 接近對照組 B，並降低 C60 與 C100 的重複批改波動。在本研究實驗環境下，V4 的批改時間約為對照組 D 的一半，顯示異質模型配置亦能改善整體執行效率。最後，實驗組 V4 與對照組 B 之比較結果顯示，V4 在 C60、C80 與 C100 中的平均分數分別與對照組 B 相差 0.96 分、0.34 分及 2.83 分，三種成績區間的差距皆未超過 3 分。在大型語言模型 Token 使用量方面，V4 於三種成績區間分別減少約 87.4%、81.8% 及 90.0%；合計由對照組 B 的 201,976 個 Token 降低至 27,725 個 Token，整體減少率約為 86.3%。雖然

V4 在 C80 與 C100 中的 MAD 仍高於對照組 B，顯示其重複批改穩定度尚未全面達到大型語言模型單一配置之水準，但其已能在大幅降低大型語言模型使用量的情況下，維持相近的平均評分結果。

綜合而言，單一小型語言模型直接取代大型語言模型，或僅將評分任務拆分為三層，皆不足以同時兼顧評分接近程度、重複批改穩定度與執行效率。相較之下，本論文提出之「小型語言模型異質分層與大型語言模型仲裁協作架構」，透過任務拆解、適性模型配置及選擇性大型語言模型仲裁，使主要評分工作得以由小型語言模型執行，並僅於雙次批改結果出現分歧時使用大型語言模型。實驗結果顯示，本論文架構僅使用約 13.7% 的大型語言模型 Token，即可維持與大型語言模型單一配置相差 3 分以內的平均評分結果。因此，本研究認為該架構已初步達成降低大型語言模型使用成本之研究目標。

五、結論與未來展望

本研究提出「小型語言模型異質分層與大型語言模型仲裁協作架構」，將程式測驗自動評分工作拆分為行為層、規範層與語言層，並依各層任務特性，分別交由訓練方向較為相符之小型語言模型執行初步評分，包括行為層使用 Qwen2.5-Coder、規範層使用 Gemma 4 E4B，以及語言層使用 Phi-4。當雙次批改結果出現分歧時，系統才交由大型語言模型 Claude Sonnet 5 進行仲裁與最終計分整合。

實驗結果顯示，當單一小型語言模型同時承擔全部評分工作時，容易因需處理多種性質不同的判斷任務，而出現跨層混淆、虛構不存在之評分標準，以及系統性偏誤等情形。相較之下，本論文所提出之架構透過任務拆解與異質模型配置，使各模型僅需處理與自身能力特性相符之評分層級。同時，該架構在維持與大型語言模型單一配置相近之平均評分結果下，可將大型語言模型 Token 使用量降低 81.8% 至 90.0%。在本研究設定與樣本下，結果顯示透過任務拆解與異質模型配置，可在維持相近平均評分結果的同時，降低大型語言模型的使用成本。然而，當前小型語言模型之發展與版本更新速度極快，本研究受限於研究時間、運算資源及消費級硬體條件，僅針對可於本地端量化部署之特定模型版本進行測試，包括 Qwen2.5-Coder、Gemma 4 E4B 與 Phi-4，尚未涵蓋所有可能適用之小型語言模型及其組合。因此，本研究所得結果應理解為在特定模型版本、量化方式與硬體環境下之實證發現，而非宣稱目前所採用之模型組合為所有任務與應用情境下的最佳配置。

未來隨著消費級硬體效能持續提升，以及小型語言模型不斷推出參數規模更小、推論速度更快且能力更強之新版本，本論文所提出之架構仍具有延伸與重新配置的彈性。後續研究可在不同硬體條件與模型版本下，重新評估各評分

層所適用之模型，並進一步比較不同異質模型組合在評分品質、穩定度、推論速度與大型語言模型使用成本上的差異。此外，各評分層所採用之推論參數（如溫度設定）如何影響評分結果與分歧偵測，本研究未進行系統性測試，亦可作為未來研究之方向。因此，本論文所提出之任務拆解、模型適性配置與選擇性大型語言模型仲裁機制，可作為未來程式測驗自動評分系統進行模型更新與架構調整之參考框架。

參考文獻

- [1] 王苙芸. (2026). 基於協作型生成式 AI 代理人架構之程式考試自動評分方法研究〔碩士論文，世新大學管理學院資訊管理學系〕.
- [2] Abdin, M., et al. (2024). “Phi-4 technical report.” arXiv preprint arXiv:2412.08905.
- [3] Anthropic. (2026). “Claude Sonnet 5 system card.” <https://www.anthropic.com/claude-sonnet-5-system-card>
- [4] Aratchige, R. M., & Ilmini, W. M. K. S. (2025). “LLMs working in harmony: A survey on the technological aspects of building effective LLM-based multi-agent systems.” arXiv preprint arXiv:2504.01963.
- [5] Ashraf, H., Danish, S. M., Leivadeas, A., Otoum, Y., & Sattar, Z. (2025). “Energy-aware code generation with LLMs: Benchmarking small vs. large language models for sustainable AI programming.” arXiv preprint arXiv:2508.08332.
- [6] Bernik, A., Radošević, D., & Čep, A. (2025). “A comparative study of large language models in programming education: Accuracy, efficiency, and feedback in student assignment grading.” *Applied Sciences*, 15(18), 10055.
- [7] Cemri, M., et al. (2025). “Why do multi-agent LLM systems fail?” In *Proceedings of the 39th Annual Conference on Neural Information Processing Systems, Datasets and Benchmarks Track*.
- [8] Chen, D., et al. (2024). “Data shunt: Collaboration of small and large models for lower costs and better performance.” In *Proceedings of the AAAI Conference on Artificial Intelligence*, 38(10).
- [9] Chen, Y., Zhao, J., & Han, H. (2025). “A survey on collaborative mechanisms between large and small language models.” arXiv preprint arXiv:2505.07460.
- [10] Cisneros-González, J., Gordo-Herrera, N., Barcia-Santos, I., & Sánchez-Soriano, J. (2025). “JorGPT: Instructor-aided grading of programming assignments with large language models (LLMs).” *Future Internet*, 17(6), 265.
- [11] Dahshan, M., Mamun, Q., & Debnath, T. (2026). “SWARM-LLM: Collaborative inference for edge-based small language models.” arXiv preprint arXiv:2606.14711.
- [12] Gemma Team, et al. (2024). “Gemma 2: Improving open language models at a practical size.” arXiv preprint arXiv:2408.00118.
- [13] Gunasekar, S., et al. (2023). “Textbooks are all you need.” arXiv preprint arXiv:2306.11644.

- [14] Hong, S., et al. (2024). “MetaGPT: Meta programming for a multi-agent collaborative framework.” In *Proceedings of the International Conference on Learning Representations (ICLR)*.
- [15] Hui, B., et al. (2024). “Qwen2.5-Coder technical report.” arXiv preprint arXiv:2409.12186.
- [16] Jukiewicz, M. (2025). “A systematic comparison of large language models for automated assignment assessment in programming education: Exploring the importance of architecture and vendor.” arXiv preprint arXiv:2509.26483.
- [17] Li, G., et al. (2023). “CAMEL: Communicative agents for ‘mind’ exploration of large language model society.” *Advances in Neural Information Processing Systems*, 36.
- [18] Messer, M., Brown, N. C. C., Kölling, M., & Shi, M. (2024). “Automated grading and feedback tools for programming education: A systematic review.” *ACM Transactions on Computing Education*, 24(1), 10:1–10:43.
- [19] Nagakalyani, G., Chaudhary, S., Apte, V., Ramakrishnan, G., & Tamilselvam, S. (2025). “Design and evaluation of an AI-assisted grading tool for introductory programming assignments: An experience report.” In *Proceedings of the 56th ACM Technical Symposium on Computer Science Education*.
- [20] Ong, I., et al. (2024). “RouteLLM: Learning to route LLMs with preference data.” arXiv preprint arXiv:2406.18665.
- [21] Pathak, A., et al. (2025). “Rubric is all you need: Improving LLM-based code evaluation with question-specific rubrics.” In *Proceedings of the 2025 ACM Conference on International Computing Education Research, Vol. 1 (ICER '25)*.
- [22] Piscitelli, A., De Rosa, M., Fuccella, V., & Costagliola, G. (2025). “Large language models for student code evaluation: Insights and accuracy.” In *Proceedings of the 17th International Conference on Computer Supported Education (CSEDU)*.
- [23] Piscitelli, A., De Rosa, M., Fuccella, V., & Costagliola, G. (2026). “LLM-based rubric grading for programming assignments: An empirical study.” In *Proceedings of the 18th International Conference on Computer Supported Education (CSEDU)*.
- [24] Tseng, E.-Q., Huang, P.-C., Hsu, C., Wu, P.-Y., Ku, C.-T., & Kang, Y. (2024). “CodEv: An automated grading framework leveraging large language models for consistent and constructive feedback.” In *Proceedings of the IEEE International Conference on Big Data (BigData)*.
- [25] Wang, F., et al. (2024). “A comprehensive survey of small language models in the era of large language models: Techniques, enhancements, applications, collaboration with LLMs, and trustworthiness.” arXiv preprint arXiv:2411.03350.

- [26] Yang, A., et al. (2024). “Qwen2.5 technical report.” arXiv preprint arXiv:2412.15115.
- [27] ZeroJudge. (n.d.). “ZeroJudge 線上程式設計評測系統.” <https://zerojudge.tw/>
- [28] Zhang, C., et al. (2026). “Confidence-calibrated small-large language model collaboration for cost-efficient reasoning.” In *Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics (EACL)*.
- [29] Zheng, L., et al. (2023). “Judging LLM-as-a-judge with MT-Bench and Chatbot Arena.” *Advances in Neural Information Processing Systems*, 36.
- [30] Zheng, T., & Hsu, K.-T. (2026). “Grading the grader: Lessons from evaluating an agentic data analysis system.” arXiv preprint arXiv:2606.24839.

附錄 1、題型與題目範例

表 7、題型與題目範例

Q1 規則題	為計算兩個整數之和並顯示結果的應用程式編寫一個 try-catch 語句。該應用程式使用兩個名為 value1TextBox 和 value2TextBox 的 TextBox 控制元件來收集輸入，並使用名為 sumLabel 的標籤控制元件來顯示結果。
Q2 邏輯題	小明：你知道蝸牛老師是什麼星座的嗎？ 小華：他那麼龜毛，一定是處女座！ 給你蝸牛老師的生日，請問他是什麼星座的？ 星座日期 Aries 牡羊座 3/21 ~ 4/20 Taurus 金牛座 4/21 ~ 5/21 Gemini 雙子座 5/22 ~ 6/21 Cancer 巨蟹座 6/22 ~ 7/22 Leo 獅子座 7/23 ~ 8/21 Virgo 處女座 8/22 ~ 9/23 Libra 天秤座 9/24 ~ 10/23 Scorpio 天蠍座 10/24 ~ 11/22 Sagittarius 射手座 11/23 ~ 12/22 Capricorn 摩羯座 12/23 ~ 1/20 Aquarius 水瓶座 1/21 ~ 2/19 Pisces 雙魚座 2/20 ~ 3/20 輸入說明： 輸入含有兩個整數 m, d ($1 \leq m \leq 12, 1 \leq d \leq 31$)，代表生日的月和日。 輸出說明： 請依題目所提供的星座日期，輸出蝸牛老師的星座名稱。 範例輸入： 8 10 範例輸出： Leo
Q3 規則題	假設 tempTextBox 是一個 TextBox 控制項的名稱，temperature 是一個 double 變數的名稱。請撰寫一個使用其中一個 TryParse 方法的 if-else 陳述式，將

	tempTextBox 控制項的 Text 屬性轉換為 double，並將結果儲存在 temperature 變數中，並將其值顯示在訊息框中。如果轉換失敗，則在訊息框中顯示錯誤訊息。
Q4 邏輯題	給你兩個日期，問這兩個日期相差幾天。 輸入說明： 輸入有多筆測資，每筆測資有兩行，每行有三個整數依序是年、月、日。題目保證不會有不符合的測資出現。 輸出說明： 輸出兩個日期差幾天。 範例輸入： 2011 10 19 2011 10 18 範例輸出： 1
Q5 規則題	假設名為 rainfall.txt 的檔案包含一個月的降雨資料。編寫一個程式碼示例，該程式碼示例使用 while 迴圈讀取檔案，並在 ListBox 控制元件中顯示其內容，同時計算月降雨總量，顯示在 Label 中。 提示： ListBox 名稱為 rainfallListBox StreamReader reader Label 名稱為 totalRainFallLabel
Q6 邏輯題	求一個數全部質因數之和比如， $6 = 2 \times 3$，則輸出 $2 + 3 = 5$ $8 = 2 \times 2 \times 2$，則輸出 $2 + 2 + 2 = 6$ 輸入說明： 每行一個正整數，$x < 2000000000$ 輸出說明：

	對應的因數和 範例輸入： 19 32 範例輸出： 19 10
Q7 問答題	請問以下程式執行結果 <pre> private void goButton_Click(object sender, EventArgs e) { int number = 99; MessageBox.Show("The value of number is " + number); ChangeMe(number); MessageBox.Show("The value of number is " + number); } private void ChangeMe(int myValue) { myValue = 0; MessageBox.Show("In ChangeMe, myValue is " + myValue); } </pre>
Q8 邏輯題	在很多情況下，我們確實要利用程序隨機產生一個由 1-N 的組成的數列，例如：要將編號由 1-N 的學生隨機排序，要將編號由 1-N 的氣球隨機排序檢測等等... 輸入說明：

	輸入一個正整數 N ($N < 2147483647$) 輸出說明: 輸出一個由 1-N 的隨機數列, 以空格分隔 範例輸入: 3 範例輸出: 2 1 3
Q9 規則題	numberArray1 和 numberArray2 都是可存放 100 個元素的陣列。請寫出將 numberArray1 的值複製到 numberArray2 的程式碼。
Q10 邏輯題	蝸牛老師新開了一門課, 第一堂課時拿了一張白紙讓學生寫上自己喜愛的水果。現在他需要用這張紙上的水果製作一份菜單。 輸入說明: 輸入只有一行, 含有 n 個 ($1 \leq n \leq 30$) 以空白隔開的水果名稱。所有水果名稱均由字母組成。(假設不會有重複出現的相同水果) 輸出說明: 將每個水果單獨輸出於一行。 範例輸入: Apple Orange Banana Pineapple 範例輸出: Apple Orange Banana Pineapple

附錄 2、結構化題目範例

表 8、結構化題目範例

原 題 目	Q5 : (10 分) 假設名為 rainfall.txt 的檔案包含一個月的降雨資料。 編寫一個程式碼示例，該程式碼示例使用 while 迴圈讀取檔案，並在 ListBox 控制元件中顯示其內容，同時計算月降雨總量，顯示在 Label 中。 提示： ListBox 名為 rainfallListBox StreamReader reader Label 名為 totalRainFallLabel
結 構 化 後 題 目	<pre>{ "id": "Q5", "type": "撰寫程式", "points": 10, "question_category": "規則題", "answer_type": "代碼", "scoring_config": { "question_type": "規則題", "weights": { "behavioral": 0.6, "syntax": 0.4, "specification": "乘法係數" } }, "rules_structure": { "behavioral_layer": {</pre>

	<pre> "focus": "檔案讀取迴圈和累加邏輯是否正確?", "total_criteria": 4, "criteria": { "①": "是否正確理解 while 迴圈的條件判斷（讀取檔案直到結束）", "②": "是否正確逐行讀取檔案內容", "③": "是否正確實現降雨數據的累加邏輯計算總量", "④": "是否正確將數據新增到 ListBox 並計算總和" } }, "specification_layer": { "focus": "有沒有完全滿足題目的所有強制要求?", "total_criteria": 6, "criteria": { "①": "必須使用 while 迴圈讀取檔案", "②": "必須使用 StreamReader reader 物件讀取", "③": "必須使用 rainfall.txt 作為檔案名稱", "④": "必須使用名為 rainfallListBox 的 ListBox 控制項顯示內容", "⑤": "必須使用名為 totalRainFallLabel 的 Label 顯示總降雨量", "⑥": "必須計算並顯示月降雨總量" } }, "syntax_layer": { "focus": "C#程式碼語法是否完全正確?", </pre>
--	--

	<pre> "total_criteria": 6, "criteria": { "①": "while 迴圈語法正確 (while (...) { }) ", "②": "StreamReader 物件建立和使用語法正確", "③": "ReadLine()方法呼叫和返回值檢查語法正確", "④": "ListBox.Items.Add()方法呼叫語法正確", "⑤": "Label.Text 屬性賦值語法正確", "⑥": "檔案關閉 (Close()或 using) 語法正確" } }, "keywords": ["while 迴圈", "StreamReader", "檔案讀取", "ListBox", "Label", "rainfall.txt", "rainfallListBox", "totalRainFallLabel", "降雨量", "月總量", "累加", "ReadLine", "Items.Add", "檔案關閉", "迴圈條件", "讀取檔案", "顯示內容", "計算總量"] </pre>
--	---

附錄 3、各模型提詞設計

SLM1 行為層

評分層定義：

- 判斷層次：行為層（Behavior）

- 評分面向：語意與邏輯

- 核心判斷問題：程式「實際做了什麼」是否正確。

- 判斷內容：

* 功能是否完整。

* 邏輯是否正確。

* 是否處理邊界條件。

- 僅依據程式實際行為判斷，不考慮是否遵循題目指定寫法，也不考慮語法、型別、編譯是否正確。

評分規則：

1. 僅依據上述「行為層」定義評分。
2. 逐項比對每個 criterion：完全符合=1，不符合、部分符合=0。
3. 每個 criterion 必須只依據程式實際行為是否達成，不得引用規範層或語言層理由。
4. 必須先在 "analysis" 中逐項寫下每個 criterion 的推理過程，再產生分數。
5. 只輸出以下 JSON，嚴禁任何其他文字、解釋或 Markdown 標籤：

```

{{
  "question_id": "...",

  "layer": "behavioral",

  "analysis": "在這裡逐項推導每個行為準則與程式碼控制流的比對過程...",

  "criteria_check": [

    {{

      "criterion": "①",

      "status": 0

    }}

  ],

  "total_criteria": N,

  "met_criteria": M,

  "behavioral_score": "M/N"

}}

"""

```

SLM2 規範層

評分層定義：

- 判斷層次：規範層（Specification）

- 評分面向：規則與指令遵循

- 核心判斷問題：是否「照題目要求的方式」撰寫。

- 判斷內容：

* 是否使用指定控制項。

* 是否符合指定格式。

* 是否遵循題目要求的撰寫方式。

* 是否違反限制條件。

- 僅依據題目要求的規範判斷，不考慮程式功能是否正確，也不考慮語法、型別、編譯是否正確。

評分規則：

1. 僅依據上述「規範層」定義評分。

2. 逐項比對每個 criterion：完全符合=1，不符合、部分符合=0。

3. 每個 criterion 必須只依據是否符合題目規範判斷，不得引用行為層或語言層理由。

4. 必須先在 "analysis" 中逐項寫下每個 criterion 的推理過程，再產生分數。

5. 只輸出以下 JSON，嚴禁任何其他文字、解釋或 Markdown 標籤：

```
{{
```

```
"question_id": "...",
```

```

"layer": "specification",

"analysis": "在這裡逐項推導每個規範準則與程式碼結構、命名或型態的比
對過程...",

"criteria_check": [

    {{

        "criterion": "①",

        "status": 0

    }}

],

"total_criteria": N,

"met_criteria": M,

"specification_score": "M/N"

}}

""

```

SLM3 語言層

評分層定義：

- 判斷層次：語言層（Syntax）
- 評分面向：語法與運算結構
- 核心判斷問題：程式是否「能正常執行」或「學生答案文字是否指出正確

輸出」。

- 判斷內容：

* 是否存在語法錯誤。

* 是否存在型別錯誤。

* 是否缺少必要的基本程式結構。

* 運算式、條件式、迴圈、方法呼叫與變數使用是否具備合法結構。

- 僅依據程式語法、型別與基本運算結構判斷，不考慮程式功能或邏輯是否正確，也不考慮是否依照題目指定方式撰寫。

評分規則：

1. 僅依據上述「語言層」定義評分。
2. 逐項比對每個 criterion：完全符合=1，不符合、部分符合=0。
3. 每個 criterion 必須只依據語法、型別與基本程式結構判斷，不得引用行為層或規範層理由。
4. 若程式邏輯錯誤，但語法與型別合法，不能僅因邏輯錯誤而判定語言層不符合。
5. 若未使用題目指定的控制項、方法或格式，但程式結構本身合法，不能僅因未遵循指定寫法而判定語言層不符合。
6. 必須先在 "analysis" 中逐項寫下每個 criterion 的判斷依據，再產生後續分數。
7. 只輸出以下 JSON，嚴禁任何其他文字、解釋或 Markdown 標籤：

8. 如果學生提供的答案是『預期輸出結果』的文字描述，根據該描述判斷語法、型別與基本程式結構是否正確，並在 "analysis" 中說明理由。

```
{{  
  
  "question_id": "...",  
  
  "layer": "syntax",  
  
  "analysis": "在這裡逐項推導每個語言層準則，檢查語法、型別、運算結構  
與基本程式結構是否正確...",  
  
  "criteria_check": [  
  
    {{  
  
      "criterion": "①",  
  
      "status": 0  
  
    }}  
  
  ],  
  
  "total_criteria": N,  
  
  "met_criteria": M,  
  
  "syntax_score": "M/N"  
  
}}
```

""

LLM 仲裁

""你是 C# 程式評量仲裁者。

底層模型對學生答案產生分數分歧。你必須依據官方準則與學生原始碼，對每個爭議層級進行獨立重新評分。

【核心原則】

1. 不得直接採納 Round 1 或 Round 2；兩者的分數與理由僅供定位爭議。
2. 必須逐項重新比對官方 criterion。
3. 每項 criterion：完全符合=1；部分符合、不符合或無法由程式碼證明=0。
4. 各層必須完全隔離，不得使用其他層級的理由給分或扣分。
5. 只評估爭議層級：{output_layers}。
6. 所有爭議層級都必須輸出，不得遺漏，也不得新增非爭議層級。
7. total_criteria 必須等於該層官方 criterion 數量。
8. met_criteria 必須等於 criteria_check 中 status=1 的數量。
9. score 必須等於 "met_criteria/total_criteria"。

【問答題特殊評分】

此題型無程式碼,僅有學生程式執行結果的「文字描述」。

請根據學生的描述,評價是否：

1. 正確理解了程式的行為
2. 準確描述了預期輸出

3. 格式符合題目要求

【三層判斷界線】

- behavioral：功能、邏輯、控制流程、輸出結果、邊界條件。
- specification：指定控制項、指定方法、格式、命名、撰寫方式、限制條件。
- syntax：語法、型別、運算式、方法呼叫、變數宣告及基本程式結構。

【題號】

{q_id}

【爭議層級與官方準則】

{criteria_context}

【學生提交的 C# 原始碼】

{student_ans}

【兩次評分的分歧資訊】

{conflict_context}

【輸出要求】

只輸出一個合法 JSON 物件，不得輸出 Markdown、前言、後記或其他文字。

criteria_check 必須依照官方準則順序輸出。

reason 只寫該 criterion 最核心的程式碼證據，不得引用其他層級理由。

reasoning 用一句話總結該層最主要的仲裁原因。

輸出格式：

```
{  
  {  
    "{q_id}": {  
      "實際爭議層級名稱": {  
        "criteria_check": [  
          {  
            "criterion": "①",  
            "status": 0,  
            "reason": "簡短且限於本層的判斷證據"  
          }  
        ],  
        "total_criteria": 1,  
        "met_criteria": 0,  
        "score": "0/1",  
        "reasoning": "該層仲裁結果的一句話總結"  
      }  
    }  
  }  
}
```